Project Extended Physicis Informed Neural Network

Antoine Levrat

05/2026

1 Project Goals

The goal of this project is to develop a complete understanding of machine learning, especially Physics Informed Neural Networks and it's Extended version. This has been done with a bibliography phase first, and then an implementation phase. This report aims to show the different progress and results of the implementations.

This study focuses on the Brinkman equation, aiming to develop a neural solver capable of reconstructing velocity and pressure fields from targeted reference datasets. The reference data is generated via a Finite Element Method (FEM) solver incorporating a highly discontinuous, non-constant permeability coefficient. This spatial discontinuity creates a sharp interface within the porous medium, introducing severe convergence challenges for standard PINNs and thus motivating the implementation of an Extended PINN (XPINN) architecture.

Important steps of implementation :

— FEM solver for Brinkman using fenics to have a good reference solution and have a good understanding of the physics in place.
— Easy PINN model on Heat equation to make sure the logic is understood.
— Adapt said model to solve Brinkman equation
— Implement from the ground up a XPINN model

2 Discovering the physics

The Brinkman equation is an essential extension of Darcy's Law. It models the flow of a viscous fluid through a high-porosity medium. For an incompressible fluid in a steady state, it is expressed as find (p, u) in $(L^2(\Omega), H^1(\Omega)^2)$:

$$\begin{cases} -\nabla p + \mu_e \nabla^2 \mathbf{u} - \frac{\mu}{K}\mathbf{u} = 0 & \in \Omega \\ \nabla u = 0 & \in \Omega \end{cases} \tag{1}$$

Where :

— $\mathbf{u}$ (Velocity) : $\mathrm{m} \cdot \mathrm{s}^{-1}$
— p (Pressure) : Pa
— μ (Dynamic Viscosity) : $\mathrm{Pa} \cdot \mathrm{s}$
— μ_e (Effective Viscosity) : $\mathrm{Pa} \cdot \mathrm{s}$
— K (Permeability) : m^2

It's main use cases is for transition zones between Navier-Stokes and Darcy flow. Here we have chosen to study it on $[0,1]^2$ with $u(x,0) = u(x,1) = \begin{pmatrix}0\\0\end{pmatrix}$, $u(0,y) = \begin{pmatrix}1\\0\end{pmatrix}$ and $p(1,y) = 0$ Hereby forcing the inlet speed to be in the x axis, and the speed at the border to be null. Pressure is fixed at the outlet. the well-posedness of the system is assumed following standard theory

3 FEM solver

Creating the FEM solver is a task made harder since this is a coupled system (p is a scalar and u a vector hence the slight difficulty). FEM solver is done on python using Fenics.

$\forall (q,v) \in (L^2(\Omega), H^1(\Omega)^2)$, the FV of the system is given by :

$$-\int_\Omega \mu_\epsilon \nabla \mathbf{u} : \nabla \mathbf{v}\, dx + \int_\Omega \frac{\mu}{K}\mathbf{u}\cdot\mathbf{v}\, dx + \int_\Omega p(\nabla\cdot\mathbf{v})\, dx - \int_\Omega u(\nabla\cdot\mathbf{q})\, dx = \int_{\partial\Omega}(\mu_\epsilon \nabla\mathbf{u}\cdot\mathbf{n} - p\mathbf{n})\cdot\mathbf{q}\, ds \tag{2}$$

We aim to simulate a fluid going inside a pipe and hiting a porous media at $x = 0.5$. To do that we have chosen K, to be a smooth transition approximation. Hence we chose :

$$K(x) = K_{porous} + (K_{free} - K_{porous}) \cdot \Phi(x)$$

Where Φ can be written as :

$$\Phi(x) = \frac{1}{2}\left(1 + \tanh\left(\frac{x - x_f}{\epsilon}\right)\right)$$

It is either -1 or 0 and in $x_f = 0.5 \pm \epsilon$ is a smooth transition.

This is done to help the future PINN devellopement and shouldn't change the final solution too much. Later calculs will prove this hypothesis. Coeficients are given here :

Table 1 – Parameters to test FEM

Coefficient	**Unité (SI)**	**Valeur numérique**
μ	Pa s	1.0
K_{porous}	m^2	1.0e-6
K_{free}	m^2	1.0e3
μ_ϵ	Pa s	1.0
p_{out}	Pa	0.0
U_{in}	$m\,s^{-1}$	1.0
ϵ	N/D	1.0e-2

This calculus was done on a mesh of 128^2 nodes to test the computation, solving took 2.795 seconds.

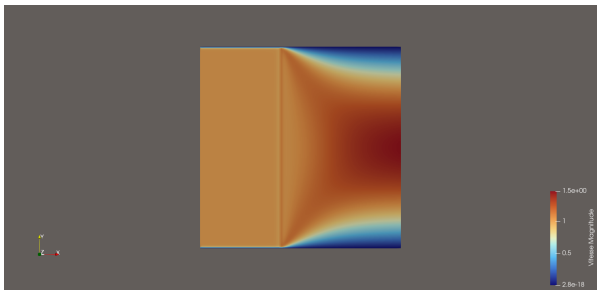

Figure 1 – Speed : u

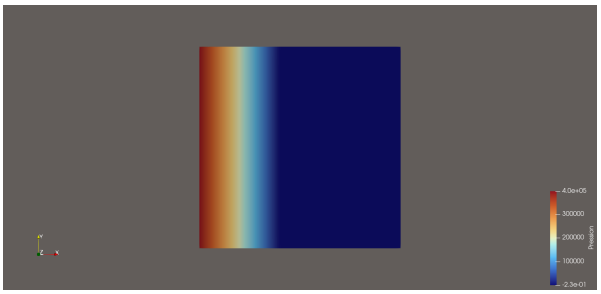

Figure 2 – pressure : p

On Figure 1, we can see that the boundary conditions are respected, both on the wall and on the inlet. On Figure 2 the outlet pressure is respected as well.

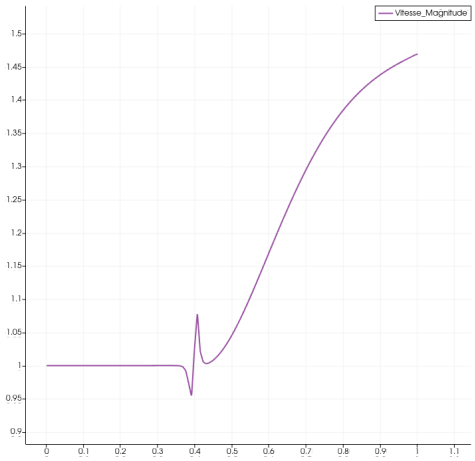

Figure 3 – speed over x axis : $u(0.5, x)$ for $x \in [0, 1]$

Figure 3 gives a good view of the speed inside the media and the different physics in place. A simple analysis is to say that first the fluid has a given speed, then it is slowed down by the frontier of the 2 medias. Then it accelerates due to the pressure in the outlet. (will not go into to much depth as understanding the physics at play is not heart of the subject of this project as of today) We also developed a small script to select few solutions points at random for the future Pinn.

4 PINN implementation

4.1 Preliminary 1D verification case

After discovering Neural newtorks and how they function in several resources such as [4], or [2], we tried to understand how to implement a solver. For this we decided to do a first run on an already functionning simple 1D code. For this we decided to implement a code from [3] and have converted it to work on a GPU.This and other test codes have allowed us to understand how to implement a PINN. Here are some visuals to show the neural network learning.

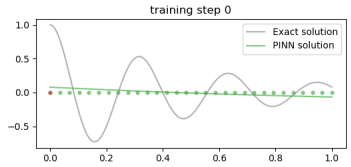

Figure 4 – PINN it 0

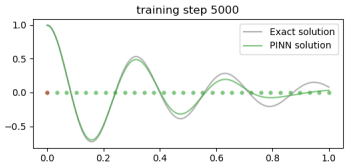

Figure 5 – PINN it 5000

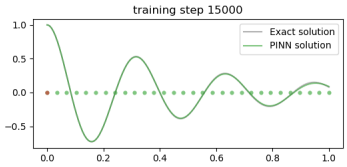

Figure 6 – PINN it 15000

We have then moved on some more complicated codes.

4.2 Brinkman PINN Solver for a constant permeability

We implemented a PINN solver using the same neural structure as in the 1D solver. Here we give the x and y coordinates and get u_x, u_y, p back where $u = \begin{pmatrix} u_x \\ u_y \end{pmatrix}$. The structure is a big larger since we the neural network has to learn more complex systems. For these first tests we kept the permeability constant in all the media.

The loss function to minimize is :

$$\mathcal{L} = w_1\mathcal{L}_{data} + w_2\mathcal{L}_{pde} + w_3\mathcal{L}_{BC}$$

where :

— $\mathcal{L}_{data} = \int_\Omega ||u - u_{data}||^2 + ||p - p_{data}||^2 \, d\Omega$

— $\mathcal{L}_{pde} = \int_\Omega || - \nabla p + \mu_e \nabla^2 \mathbf{u} - \frac{\mu}{K}\mathbf{u}||^2 + ||\nabla u||^2 d\Omega$

— $\mathcal{L}_{BC} = \int_\Omega ||u(x,0)||^2 + ||u(x,1)||^2 + ||u(0,y) - \begin{pmatrix} 1 \\ 0 \end{pmatrix}||^2 + ||p(1,0)||^2 d\Omega$

Implementation remarks :

— The weights have been adapted to make the pde count more than the 2 other factors, learning the physics seams to be a determinant factor to be able to inverse K.

— The boundary condition $u(0,y) = \begin{pmatrix} 1 \\ 0 \end{pmatrix}$ was often not respected, This is because we ask the neural network to have both a speed for all y in $x = 0$ and a null speed on $y = 0$ and $y = 1$. This was intentionnal to test the capabilities of a neural network. Putting a weight on this specific condition has drastically helped the network to converge faster.

— The fact that Pinns dont need a variationnal form was surprising at first, as most EDP solving method relly on one type of VF.

— First runs showed a big impact on cpu usage. To mitigate this effect we shifted to a GPU. This shift showed that GPUs are vastly supperior for such applications, requiering only 2% of our GPU memory compared to the 40% on our CPU.

— We started with Adam optimizer and then explored LBFGS and finaly a coupling between both. This is because Adam has been seen to lower the loss much faster in the beginning but strugeled at the end especially for converging to the value of K. In opposition, LBFGS is much more reliable at the end of the process.

Visual results Fixing K at 0.5 on all domain, The rest of the set up was the same as in the Fem application (*see 1*).

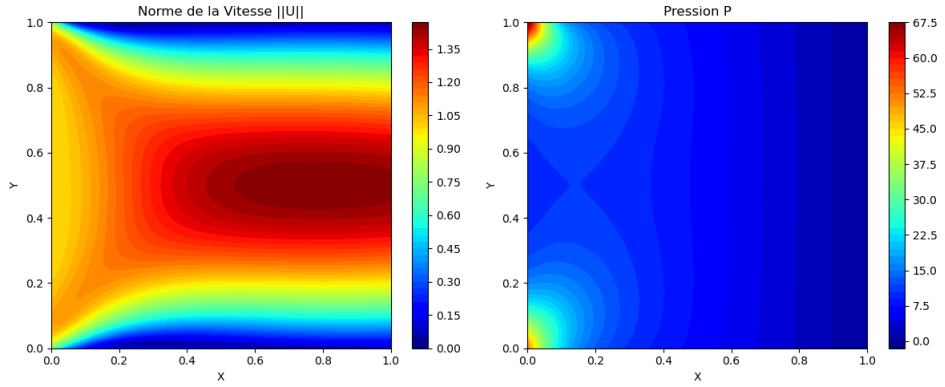

Figure 7 – speed and pressure on $[0,1]^2$ after 4000 iterations of Adam and 1200 iterations of LBFGS

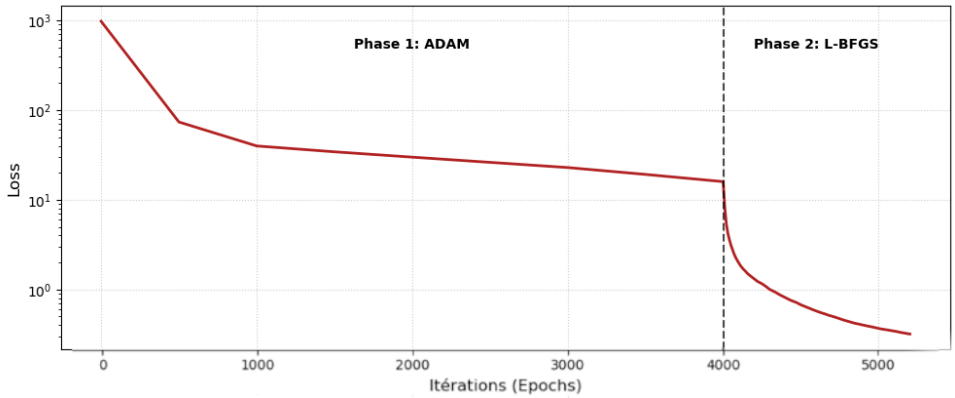

Figure 8 – Loss evolution over iterations.

Visual results in Figure 7 allow us to confirm the solver is indeed learning the physics at play. Figure 8 shows that Adam optimizer is very effective to reduce the loss in the beginning. LBFGS is in the other hand very good when loss is small and one need to precise it even more.

4.3 Brinkman PINN Solver for 2D permeability coefficient

We aimed to solve Brinkman equation as defined in the FEM section. $K(x,y)$ is now either 10^{-3} or 10^{6}. The process is strait forward, yet we encountered few issues. The network would go into local optima. adjusting weights helped, but the biggest impovement was to normalize the pressure. The pressure goes up to 10^6 and as low as 0, hence normalizing helped the network work on more stable values across all its outputs. This section has helped us to understand that stability is very important in these solvers. And that it's difficult to predict with accuracy the behaviour of the network before hand.

Tests done on this set up showed that on small Permeability discontinuities, the solver is able to converge quite well. But, our set up having 9 orders of magnitude in the permeability coefficient, this model was just not feasable even after 10^6 epochs.

Visual results : no convergence Fixing the set up as in the Fem application (*see 1*), we fall into a local optima...

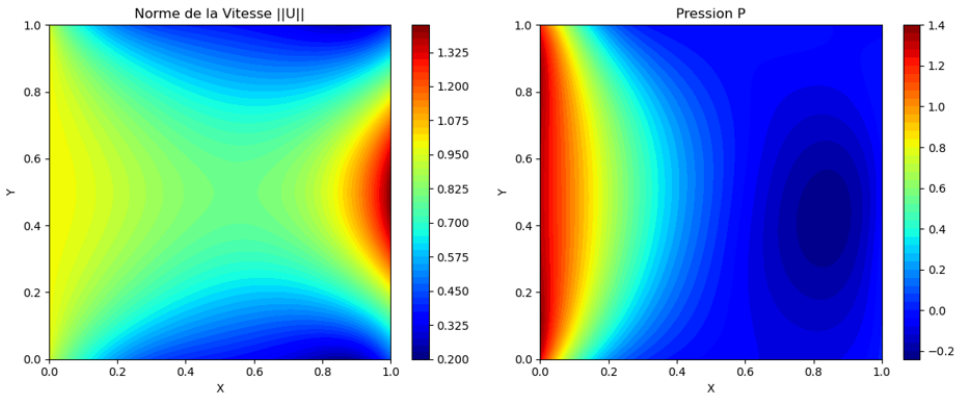

Figure 9 – speed and pressure on $[0,1]^2$ after 10000 ADAM iterations

4.4 Brinkman extended PINN Solver for 2D permeability coefficient

After reading Extended physics-informed neural networks from Ameya D. Jagtap and George Em Karniadakis [1]. This paper presents a method where domains are non-overlapping, and using the boundary points and the residual from each domains into the loss of each solver. Since we observe a clear frontier in the solution (see 2), we can define 2 neural networks for each geometrical domains.

We defined two loss functions similar to the one in [1] :

$$\mathcal{L}_1 = \alpha_1 \mathcal{L}_{data_1} + \alpha_2 \mathcal{L}_{pde_1} + \alpha_3 \mathcal{L}_{BC_1} + \alpha_4 \mathcal{L}_{u_{avg}} + \alpha_5 \mathcal{L}_R$$

where :

- $\mathcal{L}_{data_1} = \int_{\Omega_1} ||\mathbf{u}_1 - \mathbf{u}_{data}||^2 + ||p_1 - p_{data}||^2 \, d\Omega$
- $\mathcal{L}_{pde_1} = \int_{\Omega_1} \left\| -\nabla p_1 + \mu_e \nabla^2 \mathbf{u}_1 - \frac{\mu}{K_1} \mathbf{u}_1 \right\|^2 + ||\nabla \cdot \mathbf{u}_1||^2 \, d\Omega$
- $\mathcal{L}_{BC} = \int_{\partial\Omega_1} ||\mathbf{u}_1(x,0)||^2 + ||\mathbf{u}_1(x,1)||^2 + \left\| \mathbf{u}_1(0,y) - \begin{pmatrix} 1 \\ 0 \end{pmatrix} \right\|^2 \, d\Gamma$
- $\mathcal{L}_{u_{avg}} = \int_{\Gamma_I} \left\| \mathbf{u}_1 - \frac{\mathbf{u}_1 + \mathbf{u}_2}{2} \right\|^2 + \left\| p_1 - \frac{p_1 + p_2}{2} \right\|^2 \, d\Gamma$
- $\mathcal{L}_R = \int_{\Gamma_I} \left\| \left(-\nabla p_1 + \mu_e \nabla^2 \mathbf{u}_1 - \frac{\mu}{K_1} \mathbf{u}_1 \right) - \left(-\nabla p_2 + \mu_e \nabla^2 \mathbf{u}_2 - \frac{\mu}{K_2} \mathbf{u}_2 \right) \right\|^2 + ||\nabla \cdot \mathbf{u}_1 - \nabla \cdot \mathbf{u}_2||^2 \, d\Gamma$

Remarks : Boundary Conditions : Because Subdomain 1 governs only the left side of the mesh (x $\leq$ Interface), its external boundary condition only evaluates the top wall, bottom wall, and the inlet. The outlet pressure condition (p(1,0)=0) is excluded here because it belongs entirely to Subdomain 2's geometry.

Average Solution Continuity : This is your new interface term evaluating how far Subdomain 1's predictions deviate from the average prediction of both networks exactly on the common boundary.

Residual Continuity : This measures the squared difference between the Brinkman PDE residual calculated by Model 1 and the residual calculated by Model 2 along the interface, ensuring they agree on the physics at the "seam".

Implementation notes : Implementation of this method was harder than anticipated. Because of our parameters having such big magnitudes diffrences, and limited computational resources. We were, however, able to overcome these. The boundary was also

a problem that took time to understand and solve, one solution has been to make this coefficient in the loss more important in regards to the others.

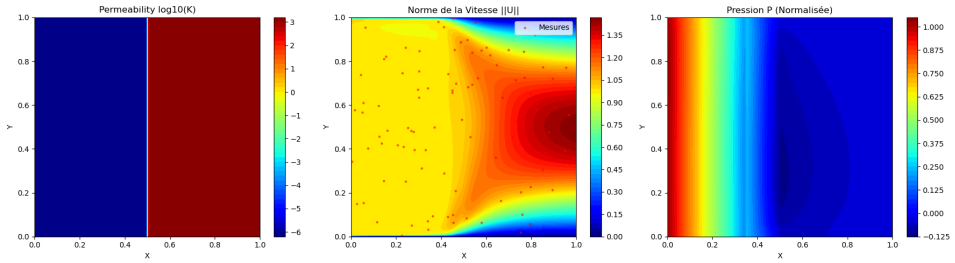

Figure 10 – xPINN, speed and pressure on $[0, 1]^2$ after 10000 ADAM iterations

Visual results : Seen in red are the data points collected from the FEM solver. One could note that the spots with less data points where the hardest for the network.

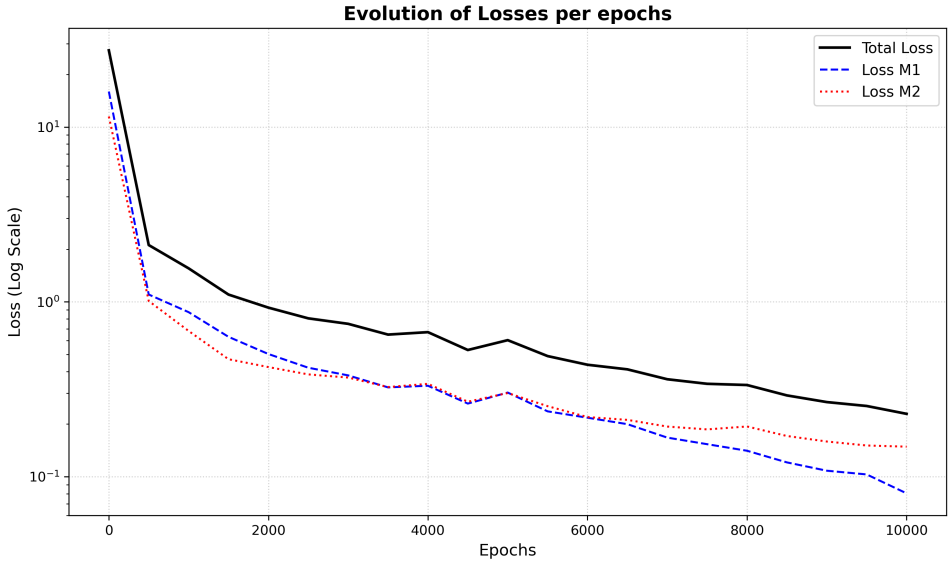

Figure 11 – xPINN, summed loss evolutions over ADAM epochs

On Figure 11, one can observe that loss converges, test would be of better accuracy if done on more iterations coupled with L-BFGS, but we were not able to do thoses computations in the time given.

5 Conclusion

This project successfully investigated the application of physics-informed neural networks to complex fluid dynamics problems. By leveraging data from a Brinkman FEM solver, we developed an xPINN framework capable of accurately reconstructing velocity and pressure fields across highly discontinuous permeability domains.

Key challenges—including loss function weight balancing, data normalization, and interface continuity conditions—were systematically addressed to guarantee convergence. The resulting multi-domain solver demonstrates strong predictive performance.

Future work will focus on extending this architecture to transient simulations, complex geometries, and the resolution of inverse problems to recover unknown permeability coefficients

Références

[1] Ameya D. Jagtap and George Em Karniadakis. Extended physics-informed neural networks (xpinns). https://ceur-ws.org/Vol-2964/article_60.pdf.

[2] Jousef Murad LITE. Physics-informed neural networks (pinns) - an introduction. https://www.youtube.com/watch?v=G_hIppUWcsc.

[3] Ben Mosley. https://github.com/stars/benmoseley/lists/sciml-workshops-lectures.

[4] Universidad of Valladolid. An introduction to neural networks. https://www.infor.uva.es/~teodoro/neuro-intro.pdf.